

전자전기컴퓨터설계실험II

실험 전 보고서

LAB6: STM32_2 UART 및 FPGA 주변장치

학과 전자전기컴퓨터공학부

이름/학번 김영서 / 2025440031

작성일 2026년 10월 26일

대상 UART 통신, 서보모터, SRAM, DAC

검증 환경

FPGA 시뮬레이터: Icarus Verilog / Vivado Simulator

보드 핀: 교재 전용 핀표 확인 후 최종 반영

1. 실험 목적 및 사전 준비 범위

1.1 실험 목적

이번 LAB6의 목적은 PC에서 입력한 한 바이트를 STM32가 수신하고, UART3를 통해 FPGA로 전달한 뒤 FPGA가 수신 문자를 에코하고 LED 명령을 수행하도록 구성하는 것이다. 이어서 FPGA의 PWM, 양방향 메모리 버스, 병렬 DAC 제어를 Verilog로 모듈화하고 자기검사 테스트벤치로 검증한다.

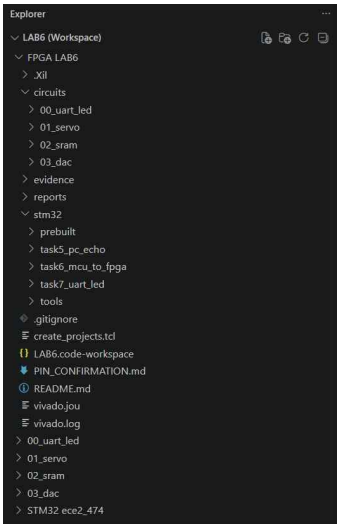


그림 1. LAB6 작업공간과 파일 구성

1.2 공통 조건

항목	조건
MCU	NUCLEO-G474RE, UART2 VCP / UART3 PC10·PC11
UART	9600 baud, 8 data, no parity, 1 stop, newline None
FPGA	HBE-Combo II-DLD Spartan-7 S75, xc7s75fgga484-1
클록·리셋	50 MHz(B6), HIGH reset(K4)
사전 검증	RTL 컴파일, self-checking PASS, VCD 생성
실험일	2026년 10월 26일

표 1. LAB6 공통 실험 조건

※ 장비 전용 핀은 추측하지 않는다. 확장 GPIO RX, 서보, SRAM, DAC 핀은 교재 핀표를 확인한 뒤 XDC의 TODO 항목에 반영한다.

2. STM32_2 과제5 · PC와 MCU 입출력

2.1 요구사항과 구현

getchar()의 결과를 int로 받아 EOF를 구분하고, 유효한 문자이면 문자와 uint8_t로 변환한 바이트 값을 16진수 두 자리로 출력한다. 입력 1, A, a의 예상값은 각각 0x31, 0x41, 0x61이다.

입력	ASCII 16진수	예상 출력
1	0x31	받은 문자: 1, 바이트 값: 0x31
A	0x41	받은 문자: A, 바이트 값: 0x41
a	0x61	받은 문자: a, 바이트 값: 0x61

표 2. 과제5 입력별 예상값



그림 2. STM32_2 과제5 문자 및 16진수 출력 코드

2.2 본 실험 확인

MCU VCP COM 포트를 9600 8N1로 열고 줄바꿈을 None으로 설정한다. 세 문자를 각각 한 번씩 보내 문자와 바이트 값이 일치하는지 확인한다. 입력 대기 상태와 오류 정지를 구분한다.

3. STM32_2 과제6·7 및 FPGA UART LED

3.1 데이터 경로



그림 3. PC–STM32–FPGA–PC UART 연결 구조

과제6은 PC→UART2 VCP→STM32→UART3 PC10→FPGA 확장 GPIO RX→FPGA USB-UART F6→PC의 흐름이다. STM32 송신 함수의 반환값 0은 함수 성공만 의미하므로 FPGA 포트에서 같은 문자가 출력되는지 별도로 확인한다. 양쪽 보드의 GND를 반드시 공통으로 연결한다.



그림 4. STM32_2 과제6 UART3 전송 코드



그림 5. STM32_2 과제7 LED 명령 전송 코드

3.2 LED 명령

문자	ASCII	LED 동작
0	0x30	끄기
1	0x31	켜기
2	0x32	현재 상태 반전
그 밖	임의	상태 유지

표 3. 과제7 UART LED 명령표

수신 유효 펄스에서 명령을 해석하고 같은 바이트를 UART TX로 에코한다. 1을 반복하면 계속 켜지고, 2를 반복하면 매번 반전되어야 한다. 리셋 시 LED는 꺼지고 TX는 HIGH 대기 상태다.

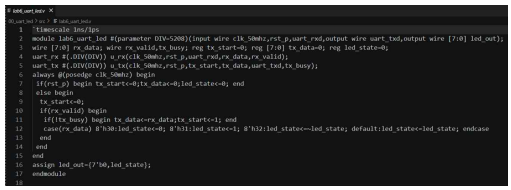


그림 6. FPGA UART 수신·에코·LED 명령 처리 코드

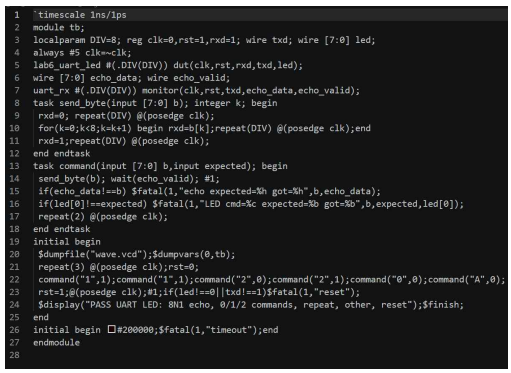


그림 7. UART LED 자기검사 테스트벤치



그림 8. UART LED 자기검사 PASS 결과



그림 9. UART 수신·에코·LED 상태 파형

3.3 XDC와 실제 연결

USB-UART 자체시험 XDC는 RX C6, TX F6, LED0 N5를 사용한다. 과제6·7에서는 RX C6를 그대로 쓰지 않고, 교재에서 선택한 확장 GPIO의 FPGA 패키지 핀으로 바꾼다. MCU PC10은 FPGA 패키지 핀 번호가 아니므로 XDC에 PC10이라고 적지 않는다.

4. FPGA 서보모터 제어

4.1 회로 구조와 예상값

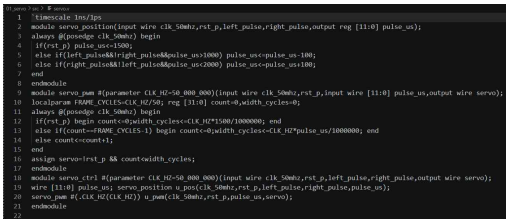


그림 10. 서보 위치값과 20 ms PWM 생성 구조

servo_position은 중앙 1500 µs에서 시작해 left/right 단일 펄스마다 100 µs씩 감소 또는 증가한다. 범위는 1000~2000 µs이며 두 입력이 동시에 들어오면 상태를 유지한다. servo_pwm은 50 Hz, 즉 20 ms 프레임을 만들고 다음 프레임 경계에서 새 펄스 폭을 적용한다.

상태	HIGH 폭	20 ms 대비 듀티
최소	1.0 ms	5.0%
중앙	1.5 ms	7.5%
최대	2.0 ms	10.0%

표 4. 서보 PWM 예상값



```
`timescale 1ns/1ps
module servo_position(input wire clk_50mhz,rst_p,left_pulse,right_pulse,output reg [11:0] pulse_us);
always @(posedge clk_50mhz) begin
 if(rst_p) pulse_us<=1500;
 else if(left_pulse&&!right_pulse&&pulse_us>1000) pulse_us<=pulse_us-100;
 else if(right_pulse&&!left_pulse&&pulse_us<2000) pulse_us<=pulse_us+100;
end
endmodule
module servo_pwm #(parameter CLK_HZ=50_000_000)(input wire clk_50mhz,rst_p,input wire [11:0] pulse_us,output wire servo);
localparam FRAME_CYCLES=CLK_HZ/50; reg [31:0] count=0,width_cycles=0;
always @(posedge clk_50mhz) begin
 if(rst_p) begin count<=0;width_cycles<=CLK_HZ*1500/1000000; end
 else if(count==FRAME_CYCLES-1) begin count<=0;width_cycles<=CLK_HZ*pulse_us/1000000; end
 else count<=count+1;
end
assign servo=!rst_p && count<width_cycles;
endmodule
module servo_ctrl #(parameter CLK_HZ=50_000_000)(input wire clk_50mhz,rst_p,left_pulse,right_pulse,output wire servo);
wire [11:0] pulse_us; servo_position u_pos(clk_50mhz,rst_p,left_pulse,right_pulse,pulse_us);
servo_pwm #(.CLK_HZ(CLK_HZ)) u_pwm(clk_50mhz,rst_p,pulse_us,servo);
endmodule
```

그림 11.서보 위치·PWM·최상위 설계 코드



그림 12. 서보 자기검사 PASS 결과



그림 13. 1000·1500·2000 µs 서보 파형

4.2 실제 보드 확인

전원과 접지를 교재 방식으로 연결하고 신호선만 FPGA 서보 출력 핀에 연결한다. 서보모터 전류를 FPGA I/O 핀에서 공급하지 않는다. 중앙·좌측 한계·우측 한계의 이동과 상·하한 고정을 영상으로 확인한다.

5. FPGA SRAM 읽기·쓰기

5.1 버스 제어

쓰기 요청만 활성화되면 FPGA가 하위 4비트를 데이터 버스에 구동하고 CS와 WE를 LOW로 만든다. 읽기 요청만 활성화되면 FPGA 출력은 Z가 되고 SRAM이 버스를 구동하며 하위 4비트를 LED에 표시한다. 읽기와 쓰기가 동시에 요청되면 세 제어신호를 비활성 상태로 두어 버스 충돌을 막는다.

ctrl_w	ctrl_r	동작	CS/OE/WE
0	0	대기, FPGA bus=Z	1/1/1
1	0	쓰기, FPGA가 bus 구동	0/1/0
0	1	읽기, FPGA bus=Z	0/0/1
1	1	충돌 차단	1/1/1

표 5. SRAM 제어 진리표



그림 14. SRAM 버스 제어 및 상위 모듈 코드



그림 15. SRAM 자기검사 PASS 결과

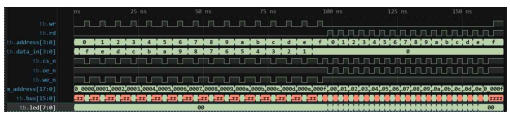


그림 16. SRAM 16주소 쓰기·읽기·Z 버스 파형

5.2 실제 보드 확인

주소 0~15에 서로 다른 값을 쓴 뒤 같은 주소를 읽어 LED 값과 비교한다. 쓰기 중 OE 비활성, 읽기 중 WE 비활성, 대기 중 데이터 버스 Z를 확인한다. SRAM 전용 핀은 교재 표에 따라 모두 지정해야 비트스트림 생성이 가능하다.

6. FPGA DAC

6.1 램프와 병렬 쓰기

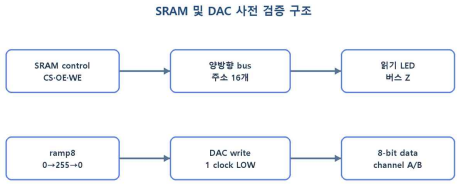


그림 17. SRAM 버스와 DAC 병렬 출력의 사전 검증 구조

ramp8은 설정된 주기마다 8비트 값을 1씩 증가시키며 255 다음에는 0으로 돌아간다. dac_parallel8은 update 시 데이터와 채널을 등록하고 WE를 한 클록 동안 LOW로 만든다. 데이터와 채널은 쓰기 펄스가 끝날 때까지 유지된다.

항목	예상 동작
데이터	0→1→…→255→0
CS_n / LDAC_n	연속 LOW
WE_n	update마다 1클록 LOW
A/B	add_sel 값을 쓰기 시점에 등록
LED	dac_d와 동일

표 6. DAC 디지털 인터페이스 예상값



그림 18. DAC 램프·병렬 쓰기·최상위 코드

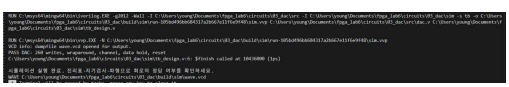


그림 19. DAC 자기검사 PASS 결과

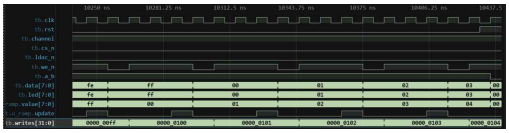


그림 20. DAC FE→FF→00 랩어라운드 파형

6.2 실제 보드 확인

교재 핀표로 DAC 제어선과 D[7:0]를 지정하고 아날로그 출력은 멀티미터 또는 오실로스코프로 측정한다. 디지털 램프가 증가할수록 출력 전압이 단계적으로 증가하고 255 다음 0에서 되돌아가는지 확인한다.

7. 본 실험 진행 계획

순서	예정 작업	확인할 내용
1	장비 교재 핀표 확인	확장 GPIO·서보·SRAM·DAC 실제 핀
2	XDC 작성 및 검토	모든 최상위 포트와 I/O 규격
3	기능 시뮬레이션 재확인	예비 단계 PASS·파형과 비교
4	합성·구현	오류와 타이밍 위반 여부
5	비트스트림 생성·다운로드	대상 FPGA와 파일 확인
6	보드 시연	UART·서보·SRAM·DAC 실제 동작
7	결과 증빙	사진·영상·계측값 확보

표 7. LAB6 본 실험 예정 절차

본 실험에서는 장비 교재의 핀 배치표를 먼저 확인하여 XDC를 작성하고, 기능 시뮬레이션 결과를 재확인한 뒤 합성, 구현, 비트스트림 생성 및 실제 보드 시연을 순서대로 수행할 예정이다. 예비 단계에서는 실제 Vivado 실행 결과를 보고서 증빙으로 사용하지 않는다.

8. 사전 검증 결과 및 체크리스트

회로	self-checking 결과	파형
UART LED	PASS: 에코·0/1/2·반복·기타·리셋	wave.vcd 생성
서보	PASS: 20 ms·1/1.5/2 ms·한계·동시 입력	wave.vcd 생성
SRAM	PASS: 16주소·Z·동시요청	wave.vcd 생성
DAC	PASS: 260회·255→0·채널·hold	wave.vcd 생성

표 8. LAB6 FPGA 사전 시뮬레이션 결과

실험 전 준비 완료 항목은 STM32 과제5~7 독립 코드, FPGA 네 회로 RTL, 자기검사 테스트벤치, VS Code 실행 작업과 장비 핀 확인표다. 실제 XDC 완성, 장치 연결, 합성·구현과 계측 결과는 본 실험 후 결과보고서에 기록한다.

9. GitHub 자료 링크

아래 주소는 예비보고서에 사용한 소스와 검증 자료를 기준 커밋에 고정한 GitHub 원문 주소이다. 주소 전체를 그대로 기재하였으므로 main 브랜치가 변경되어도 같은 자료를 확인할 수 있다.

9.1 저장소 및 기준 버전

• 저장소 URL:
https://github.com/dalkomys777/fpga_lab6

• 기준 브랜치: main
• 증빙 기준 태그: prelab-evidence-v1.0.0
• 증빙 기준 커밋:
2d2ca03526d2908695ce478989e4f41778d8edef

• 증빙 커밋 주소:
https://github.com/dalkomys777/fpga_lab6/commit/2d2ca03526d2908695ce478989e4f41778d8edef

• 기준 태그 주소:
https://github.com/dalkomys777/fpga_lab6/tree/prelab-evidence-v1.0.0

• 사용자 캡처 폴더:
https://github.com/dalkomys777/fpga_lab6/tree/2d2ca03526d2908695ce478989e4f41778d8edef/evidence/user_captures

• 예비 검증 자료 폴더:
https://github.com/dalkomys777/fpga_lab6/tree/2d2ca03526d2908695ce478989e4f41778d8edef/evidence/prelab

• 프로젝트 설명:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/README.md

• 핀 확인 안내:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/PIN_CONFIRMATION.md

• VS Code 작업공간:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/LAB6.code-workspace

9.2 사용자 캡처 자료

• 공통 작업공간:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/evidence/user_captures/01_workspace.png

• STM32 과제5 코드:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/evidence/user_captures/02_stm32_task5.png

• STM32 과제6 코드:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/evidence/user_captures/03_stm32_task6.png

• STM32 과제7 코드:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/evidence/user_captures/04_stm32_task7.png

• UART LED 설계 코드:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/evidence/user_captures/05_

uart_led_code.png

• UART LED PASS 결과:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/evidence/user_captures/06_uart_pass.png

• UART LED 파형:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/evidence/user_captures/07_uart_wave.png

• UART LED 테스트벤치:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/evidence/user_captures/08_uart_testbench.png

• 서보 설계 코드:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/evidence/user_captures/09_servo_code.png

• 서보 PASS 결과:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/evidence/user_captures/10_servo_pass.png

• 서보 파형:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/evidence/user_captures/11_servo_wave.png

• SRAM 설계 코드:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/evidence/user_captures/12_sram_code.png

• SRAM PASS 결과:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/evidence/user_captures/13_sram_pass.png

• SRAM 파형:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/evidence/user_captures/14_sram_wave.png

• DAC 설계 코드:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/evidence/user_captures/15_dac_code.png

• DAC PASS 결과:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/evidence/user_captures/16_dac_pass.png

• DAC 파형:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/evidence/user_captures/17_dac_wave.png

9.3 소스 코드 및 테스트벤치

• STM32 과제5 소스:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/stm32/task5_pc_echo/ece2_474_routine.c

• STM32 과제6 소스:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/stm32/task6_mcu_to_fpga/ece2_474_routine.c

• STM32 과제7 소스:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/stm32/task7_uart_led/ece2_474_routine.c

• UART 송수신 모듈:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/circuits/00_uart_led/src/uart.v

• UART LED 상위 모듈:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/circuits/00_uart_led/src/lab6_uart_led.v

• UART LED 테스트벤치:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/circuits/00_uart_led/sim/tb_design.v

• 서보 설계 모듈:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/circuits/01_servo/src/servo.v

• 서보 테스트벤치:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/circuits/01_servo/sim/tb_design.v

• SRAM 설계 모듈:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/circuits/02_sram/src/sram.v

• SRAM 테스트벤치:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/circuits/02_sram/sim/tb_design.v

• DAC 설계 모듈:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/circuits/03_dac/src/dac.v

• DAC 테스트벤치:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/circuits/03_dac/sim/tb_design.v

9.4 XDC 제약 파일

• UART LED XDC:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/circuits/00_uart_led/constraints/lab6_uart_led.xdc

• UART MCU RX XDC 템플릿:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/circuits/00_uart_led/constraints/lab6_uart_led_mcu_rx_template.xdc

• 서보 XDC:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/circuits/01_servo/constraints/lab6_servo.xdc

• SRAM XDC:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/circuits/02_sram/constraints/lab6_sram.xdc

• DAC XDC:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/circuits/03_dac/constraints/lab6_dac.xdc

9.5 시뮬레이션 원본 파형

• UART LED VCD:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/evidence/prelab/00_uart_led/wave.vcd

• 서보 VCD:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/evidence/prelab/01_servo/wave.vcd

• SRAM VCD:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/evidence/prelab/02_sram/wave.vcd

• DAC VCD:
https://github.com/dalkomys777/fpga_lab6/blob/2d2ca03526d2908695ce478989e4f41778d8edef/evidence/prelab/03_dac/wave.vcd

• 재현 방법: git checkout prelab-evidence-v1.0.0 후 LAB6.code-workspace를 열고 각 회로의 시뮬레이션 작업을 실행한다.

• 예비보고서 PDF:
https://github.com/dalkomys777/fpga_lab6/blob/main/reports/pre/예비리포트_LAB6_2025440031김영서.pdf

10. 참고문헌

[1] ece2_474/docs/STM32_2.md, “컴퓨터와 FPGA 통신”, 2026-10-26 실습 안내.
[2] ece2_474/docs/FPGA_additional.md, “Servo·SRAM·DAC 설계 모듈 및 테스트벤치”.
[3] AMD, Vivado Design Suite User Guide: Using Constraints (UG903).
[4] ARM, Cortex-M4 Devices Generic User Guide, exception and interrupt handling.